

Harness Teardown

Day 1 · after Module 2 (Agentic Engineering) · 25 min · pods of 3–5 · whiteboard / on paper

Objective. Every pod gets the same one-page agent harness. Find the failure modes, **circle the single one you'd fix first**, write the smallest fix that addresses it, then say which flaw bites **your pod's case** hardest.

Reminder: the harness is the plumbing around the model — the loop, the tools, the checkpoints, the logging. Most of what goes wrong with an agent lives here, not in the model's answers.

The harness

```
1  # Meridian Agent Runtime v0.1 — shared harness sketch
2  agent  = Agent(model="meridian-llm", system_prompt=SYSTEM)
3  tools  = load_all_tools()                # every registered tool
4  context = [SYSTEM, user_request]
5
6  while True:                             # main loop
7      response = agent.run(context, tools)
8      if response.tool_calls:
9          for call in response.tool_calls:
10             result = call.execute()      # runs immediately
11             context.append(result.raw)   # raw output back into context
12         else:
13             context.append(response.text)
14         # ... loop continues ...
15
16 # registered tools: read_file, web_fetch, send_email,
17 # post_ledger_entry, execute_payment, run_sql, shell
18
19 def execute(call):
20     try:
21         return tool.invoke(call.args)
22     except:
23         pass                            # on error, keep going
24
25 # SYSTEM prompt contains API_KEY and DB_PASSWORD
26 # logging: none
27 # result returned to user only when the loop ends
```

Your task (≈12 min)

- Mark every flaw you can find. Note the line number.
- Circle the **one you'd fix first** and finish the sentence: *"Most dangerous because..."*
- Write the **smallest fix** for it — one line, not a redesign.
- Which flaw is worst for **your pod's** use case, and why?

LINE #	THE FLAW (NAME IT)	SMALLEST FIX

Closing reflection. Look at your list. How many of these are about the *model* being wrong — and how many are about the *harness* around it? What does that ratio say about where validation effort should go?

Illustrative for the exercise only. The sketch is intentionally broken and simplified; a real runtime has more moving parts. Nothing here is a reference architecture.